

Flood Probability Classifier

Gradient Boosting Machine Model Documentation

MKM Research Labs

Version 2.2 — 30-October-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1 Executive Summary **3**

2 Model Purpose and Scope **3**
2.1 Purpose . . . 3
2.2 Scope . . . 3
2.3 Model Chain Position . . . 4

3 Mathematical Framework **4**
3.1 Model Family . . . 4
3.2 Feature Space . . . 4
3.3 Physical Motivation for Log Transformation . . . 5
3.4 Rising vs Falling Water . . . 5
3.5 Target Variable . . . 5
3.6 Training Data . . . 5
3.6.1 v2.0 (stressm): Multi-Storm Sequence Training . . . 5
3.6.2 v1.3 (stress): Single-Storm Balanced Training . . . 6

4 Hyperparameters **6**

5 Implementation Details **7**
5.1 Source Code . . . 7
5.2 Training Pipelines . . . 7
5.2.1 v2.0 stressm Pipeline . . . 7
5.2.2 v1.3 stress Pipeline (retained for single-storm Tab 7) . . . 7
5.3 Prediction Interface . . . 7
5.4 Probability Surface . . . 7
5.5 Serialisation . . . 8

6 Validation and Metrics **8**
6.1 Evaluation Metrics . . . 8
6.2 Production Results (v2.0 stressm) . . . 9
6.3 Version Comparison: v1.1 → v1.2 → v1.3 → v2.0 . . . 9
6.3.1 Asymmetry Evidence: Rising vs Falling Water . . . 9
6.4 Feature Importance . . . 10
6.5 Automated Test Results . . . 10

7 Sensitivity Analysis **10**

8 Model Selection: Challenger Comparison **12**
8.1 Open Question . . . 12
8.2 Why Random Forest Is the Natural Challenger . . . 12
8.3 Candidate Set . . . 12
8.4 Prerequisite: Remove Storm Leakage . . . 13

8.5 Accuracy Metrics 13
8.6 Stability Tests 13
8.7 Statistical Comparison 14
8.8 Decision Rule 14
8.9 Running the Benchmark 15
8.10 Results 15

9 Model Limitations and Known Weaknesses **16**

10 Change History **17**

1 Executive Summary

This document presents the mathematical framework, implementation, and validation of the Flood Probability Classifier (MKM-FC-001), a Gradient Boosting Machine (GBM) model that estimates per-gauge flood probability as a function of log-transformed water level, storm time, and hydrograph derivatives: $F\left(\ln \frac{w}{s}, \ln \frac{t+1}{T}, \Delta \ln \frac{w}{s}, \Delta^2 \ln \frac{w}{s}\right) \to P(\text{flood})$, where s is the severe flood threshold and $T = 168$ hours.

The classifier is trained per gauge using stress test vectors derived from historical storm simulations. All features are log-transformed relative to gauge-specific thresholds, producing a scale-invariant representation that generalises across gauges with different flood severity levels.

v1.3 enhancement introduced four log-transformed features: $\ln(w/s)$, $\ln((t+1)/168)$, $\Delta \ln(w/s)$, and $\Delta^2 \ln(w/s)$. This produced a mean AUC of 0.9525 across 40 gauges, a 3.2% improvement over v1.2.

v2.0 upgrade (multi-storm): Version 2.0 replaces the single-storm training corpus with sequences generated by the Storm Sequence Generator (MKM-SS-001). Instead of 1,000 independent storms ($\times$168h = 168,000 vectors per gauge), the classifier now trains on **10,000 multi-storm sequences** ($\times$168h = **1,680,000 vectors per gauge**)—ten times the data. Crucially, multi-storm hydrographs contain *multi-peak* shapes driven by compounding antecedent-wetness effects that single-storm data cannot represent. The feature space and hyperparameters are unchanged from v1.3; the gains come entirely from richer, larger training data.

The mean AUC-ROC across all 52 Thames gauges improved from 0.9525 (v1.3) to $\approx$**0.993** (v2.0), a further 4.2% improvement. The stressm classifiers are stored in `data/output/stressm/` and trained via `phys.py port --stressm --train-classifier`.

Key risk: The model is trained on synthetically noised data (time-decaying Gaussian perturbation), which creates intentional early-hour uncertainty. The derivative features are computed from the log-transformed noised hydrograph, preserving the uncertainty structure in the velocity and acceleration signals.

2 Model Purpose and Scope

2.1 Purpose

The Flood Probability Classifier serves two functions within the platform:

- **Stress testing:** Given a stress storm scenario, estimate the probability of flood alert breach at each gauge as a function of evolving water levels during the storm event.
- **P&L estimation:** Convert flood probabilities into cash-equivalent stress P&L for PRS trade positions, using the CDS-in-stress analogy: cash_price = notional $\times$ P(flood) $\times$ direction.

2.2 Scope

- **In scope:** Per-gauge binary classification (flood/no-flood) using water level, temporal, and hydrograph derivative features.
- **Out of scope:** Multi-gauge spatial correlation, flood severity estimation (depth/velocity), and climate change projection.

2.3 Model Chain Position

The classifier now sits downstream of the Storm Sequence Generator:

MKM-SI-001 → **MKM-SS-001** (sequences) → MKM-SG-001 (hydrograph) → **MKM-FC-001** (v2.0) → Stress F

For single-storm stress testing (Trading Desk Tab 7), the v1.3 classifiers stored in `data/output/stress/` remain in use. The v2.0 stressm classifiers stored in `data/output/stressm/` power the Portfolio Stress tab (Tab 8).

3 Mathematical Framework

3.1 Model Family

The model uses `sklearn.ensemble.GradientBoostingClassifier`, which constructs an additive ensemble of regression trees optimised under the binomial deviance loss:

$$\mathcal{L}(y, F) = -\sum_{i=1}^{N} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

where $p_i = \sigma(F(x_i))$ and σ is the logistic function. The model is built sequentially:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

where h_m is a regression tree fit to the negative gradient of the loss, and ν is the learning rate.

3.2 Feature Space

The model uses a four-dimensional log-transformed feature space:

Feature	Type	Description
`log_h_s`	Continuous	$\ln(\tilde{w}(t)/s)$; water level relative to severe threshold s. Zero at severe level, negative below, positive above.
`log_t_end`	Continuous	$\ln((t+1)/168)$; log storm time. Ranges from ≈ -5.1 (hour 0) to 0 (hour 167).
`delta_log_h`	Continuous	$\Delta \ln(w/s)(t)$ = `log_h_s`(t) − `log_h_s`$(t-1)$; log velocity (positive rising, negative falling).
`delta2_log_h`	Continuous	$\Delta^2 \ln(w/s)(t)$ = `delta_log_h`(t) − `delta_log_h`$(t-1)$; log acceleration.

All derivative features are computed from the log-transformed noised hydrograph $\ln(\tilde{w}(t)/s)$, not the clean signal. This ensures the model learns to interpret derivative signals that include the same observation noise present in real-time monitoring. At prediction time, raw derivatives Δw and $\Delta^2 w$ are passed in and converted to log-space internally by the predictor.

3.3 Physical Motivation for Log Transformation

The raw-level feature space has two limitations:

1. **Gauge heterogeneity:** Thames gauges have severe thresholds ranging from approximately 3 m to 7 m AOD. A water level of 4.5 m carries very different flood implications depending on the gauge's threshold. The log transform $\ln(w/s)$ normalises to a common scale: $\ln(w/s) = 0$ always means "at severe threshold", regardless of absolute level.
2. **Velocity interpretation:** A rise of 0.3 m/h from 1 m is alarming; the same rise from 5 m is less informative. Log-velocity $\Delta \ln(w/s) \approx \Delta w/w$ normalises by the current level, providing a relative rate of change. This is analogous to using log-returns in financial risk models.

The time feature $\ln((t+1)/168)$ is restored from v1.2 in log scale. This compresses the early storm hours (where uncertainty is highest) and stretches the late hours (where remaining uncertainty is lower), matching the structure of the noise model $\sigma(t)$.

3.4 Rising vs Falling Water

The log-velocity feature $\Delta \ln(w/s)$ preserves the directional asymmetry introduced in v1.2:

- **Rising water** ($\Delta \ln(w/s) > 0$): The storm has not yet peaked. $P(\text{flood})$ should reflect uncertainty about the eventual peak.
- **Falling water** ($\Delta \ln(w/s) < 0$): The storm has already peaked. If near alert level, the peak was likely above it. $P(\text{flood})$ should be higher, reflecting stronger evidence.

The log-acceleration $\Delta^2 \ln(w/s)$ further discriminates between a decelerating rise (approaching peak) and an accelerating rise (early surge).

3.5 Target Variable

Binary label $y \in \{0, 1\}$ indicating whether the storm breaches the gauge's flood alert trigger level at any point during the event. The label is computed from the stress test vector:

$$y_i = \begin{cases} 1 & \text{if } \max_t w_i(t) \geq \text{alert_level} \\ 0 & \text{otherwise} \end{cases}$$

3.6 Training Data

3.6.1 v2.0 (stressm): Multi-Storm Sequence Training

Version 2.0 trains on the full catalogue of 10,000 `StormSequence` objects generated by MKM-SS-001 (Section 2.3). Each sequence contains 1–5 individual storms with calibrated durations, inter-storm gaps, and correlated intensity factors. The training corpus is therefore the complete population, not a balanced sample.

For each sequence, a multi-peak hydrograph is synthesised at each gauge by compositing the individual storm responses in temporal order. Each storm contributes a water level arc derived from its intensity factor and the gauge's spatial response parameters (`base_level_m`, `level_change_m`, `peak_position`). Between storms, the water level decays toward baseline at the catchment drainage rate, modelling the partial recovery that determines antecedent wetness for the next storm.

The same time-decaying Gaussian noise is applied per storm arc:

$$\tilde{w}(t) = w(t) + \varepsilon(t), \quad \varepsilon(t) \sim \mathcal{N}\big(0, \sigma(t)^2\big)$$

$$\sigma(t) = 0.5 \cdot e^{-3t/167} + 0.1$$

The feature extraction is identical to v1.3; the features are computed from the full 168-hour noised multi-peak hydrograph:

$$\mathbf{x}_i = \big[\, \ell(t),\ \tau(t),\ \Delta\ell(t),\ \Delta^2\ell(t) \,\big] \in \mathbb{R}^4$$

Total training corpus: **1,680,000 vectors per gauge** (10,000 sequences × 168 hours).

3.6.2 v1.3 (stress): Single-Storm Balanced Training

For reference, the v1.3 (single-storm) training pipeline used a balanced sample of 1,000 storms from the 10,000-storm catalogue:

- **Alert storms (260):** All storms that breached the flood alert trigger, sourced from `stress_storms.json`.
- **Non-alert storms (740):** Randomly sampled from the non-alert population.

Total: 168,000 vectors per gauge (1,000 storms × 168 hours).

Why v2.0 does not need balanced sampling: The multi-storm sequences naturally contain diverse hydrograph shapes including: (i) sequences where the gauge barely rises (Storm 1 is weak, far away); (ii) sequences where Storm 1 causes a modest rise and Storm 2 produces the flood (antecedent-wetness amplification); (iii) catastrophic sequences where all storms breach the severe threshold. This diversity provides the classifier with sufficient negative examples without artificial sub-sampling.

Log-level features are computed identically to v1.3:

$$\ell(t) = \ln\left(\max\left(\frac{\tilde{w}(t)}{s},\ 10^{-8}\right)\right), \quad \Delta\ell(t) = \ell(t) - \ell(t{-}1), \quad \Delta^2\ell(t) = \Delta\ell(t) - \Delta\ell(t{-}1)$$

$$\tau(t) = \ln\left(\frac{t+1}{168}\right)$$

with boundary conditions $\Delta\ell(0) = \Delta^2\ell(0) = 0$.

4 Hyperparameters

Parameter	Value	Rationale
`n_estimators`	300	Sufficient for convergence without overfitting
`max_depth`	3	Captures water-level × hour interaction
`learning_rate`	0.05	Shrinkage paired with 300 rounds
`subsample`	0.8	Row sampling to reduce variance
`min_samples_leaf`	50	Prevents small-node overfitting
`random_state`	42	Reproducibility
`test_size`	0.2	80/20 train-test split

The stressm trainer holds out 20% of storms, stratified on whether the storm floods, so every storm is entirely in training or entirely in test (Section 8.4).

5 Implementation Details

5.1 Source Code

- **Primary module:** `src/models/stress/flood_classifier.py`
- **Training class:** `FloodClassifierTrainer`
- **Prediction class:** `FloodPredictor`

5.2 Training Pipelines

5.2.1 v2.0 stressm Pipeline

1. Generate 10,000 StormSequence objects via MKM-SS-001 (`--stressm`)
2. For each gauge, synthesise 168-hour multi-peak hydrograph per sequence
3. Apply time-decaying noise; compute 4 log-transformed feature vectors
4. Build feature matrix $\mathbf{X} \in \mathbb{R}^{1{,}680{,}000 \times 4}$ and label vector $\mathbf{y} \in \{0,1\}^{1{,}680{,}000}$
5. Skip single-class gauges
6. 80/20 train-test split by storm with `random_state=42`; near-miss hydrographs stay in training
7. Fit `GradientBoostingClassifier` on training set
8. Evaluate on test set: accuracy, AUC-ROC, Brier score, log loss
9. Serialise model to `data/output/stressm/GAUGE-{id}.joblib`
10. Write summary to `data/output/stressm/training_summary.json`

5.2.2 v1.3 stress Pipeline (retained for single-storm Tab 7)

1. Load stress test vectors from `data/input/thames/stress_vectors.json`
2. For each gauge, extract feature matrix $\mathbf{X} \in \mathbb{R}^{168{,}000 \times 4}$ and label vector $\mathbf{y} \in \{0,1\}^{168{,}000}$
3. Stratified 80/20 split; fit; evaluate; serialise to `data/output/stress/GAUGE-{id}.joblib`

5.3 Prediction Interface

Three prediction modes are provided:

- `predict(gauge_id, water_level, hour, delta_w, delta2_w)` — Single-point $\hat{p}$; `delta_w` and `delta2_w` default to 0.0 (steady state) for backward compatibility
- `predict_series(gauge_id, levels, hours, delta_ws, delta2_ws)` — Vectorised time series
- `predict_surface(gauge_id, ..., delta_w, delta2_w)` — Full (w, t) probability grid at constant derivatives (default 0.0)

5.4 Probability Surface

The `predict_surface` mode produces a two-dimensional grid of flood probabilities $P(w, t)$ over a range of water levels w and storm hours t. This is exposed in the stress test UI as the "Surface" tab, displaying a heat-map table with the following structure:

Axis	Description
Rows (Y)	Water levels in 0.5 m steps, descending from highest to lowest
Columns (X)	Storm hours at 4-hour intervals: H0, H4, H8, …, H56
Cells	$P(\text{flood})$ as a percentage to one decimal place

The surface grid is computed on-demand during stress scenario execution with $\Delta w = 0$ and $\Delta^2 w = 0$ (steady-state assumption), representing the probability at each grid point if the water level were neither rising nor falling. The hourly time series uses actual derivatives computed from the hydrograph. Water level bounds are derived from the storm hydrograph range, extended to cover the gauge's full trigger band (alert through severe + 0.5 m buffer).

Visual encoding:

- Cells below the alert level: neutral (white) background with grey intensity shading proportional to $P(\text{flood})$.
- Cells in the alert band (alert $\leq w <$ warning): amber background (`#FFF8E1`).
- Cells in the warning band (warning $\leq w <$ severe): orange background (`#FFF3E0`).
- Cells at or above the severe level: red background (`#FFEBEE`).
- Post knock-out (after the first trigger breach hour): cells are left blank with a grey background, reflecting contract termination.

Within each trigger band, cell opacity increases proportionally to the probability value, creating a heat-map effect that highlights regions of high flood risk.

5.5 Serialisation

Models are persisted using `joblib.dump()` in the standard sklearn format. Each gauge model is stored independently at `GAUGE-{id}.joblib`, enabling selective retraining without full pipeline re-execution.

6 Validation and Metrics

6.1 Evaluation Metrics

Four metrics are computed per gauge on the held-out test set:

Metric	Description
Accuracy	Fraction of correct binary predictions
AUC-ROC	Area under the Receiver Operating Characteristic curve; measures discrimination ability independent of threshold
Brier Score	Mean squared error of probabilistic predictions; lower is better
Log Loss	Negative log-likelihood of predicted probabilities; penalises confident wrong predictions

6.2 Production Results (v2.0 stressm)

Training across all 52 Thames catchment gauges with 10,000 multi-storm sequences:

- **Average AUC-ROC:** ≈0.993 (row-level split, before v2.2; see Section 8.4)
- **Representative gauge (GAUGE-8f1d368d):** AUC 0.9927, accuracy 0.9776, flood rate 8.4%
- **Training sequences:** 10,000 StormSequence objects (all types)
- **Vectors per gauge:** 1,680,000 (10,000 sequences × 168 hours)
- **Gauges:** 52 (full Thames portfolio; no gauges skipped)
- **Feature importance:** `log_h_s` ≈70%, `log_t_end` ≈24%, derivatives ≈6%

6.3 Version Comparison: v1.1 → v1.2 → v1.3 → v2.0

The following table shows the progression across all model versions:

Version	Key Change	Mean AUC	Vectors/gauge
v1.1	2 features (level, hour)	0.9189	168,000
v1.2	+ Δw, $\Delta^2 w$ (log-raw)	0.9557	168,000
v1.3	Log-transform all features	0.9525	168,000
v2.0	**Multi-storm sequences; 10× data**	**≈0.993**	**1,680,000**

Analysis: The v1.1→v1.2 jump (+4.0%) came from adding derivative features. The v1.2→v1.3 slight reduction (−0.3%) reflects the more rigorous log-space normalisation (marginally harder problem, better-calibrated probabilities). The v1.3→v2.0 step (+4.2%) comes entirely from richer, larger training data: multi-peak hydrographs force the classifier to learn antecedent-wetness dynamics, and 10× more data reduces tree estimation variance throughout the GBM ensemble.

6.3.1 Asymmetry Evidence: Rising vs Falling Water

The key diagnostic is the model's ability to distinguish rising from falling water at the same level. The following table shows $P(\text{flood})$ for gauge `GAUGE-32e88723` (AUC = 0.957, flood rate = 10.0%) at hour = 20 with $|\Delta w| = 0.4$ m/h:

Level (m)	Rising	Steady	Falling	ΔP
3.3 (sub-alert)	0.003	0.027	0.103	−0.099
3.8	0.033	0.140	0.462	−0.429
4.3 (near alert)	0.261	0.646	0.882	−0.622
4.8 (above alert)	0.831	0.942	0.976	−0.144
5.3	0.984	0.992	0.996	−0.012

The model has learned the physically correct signal:

- At sub-alert levels, **falling water implies higher $P(\text{flood})$**: if the water is at 4.3 m and falling, the storm already peaked higher, providing strong evidence that alert was breached.
- At sub-alert levels, **rising water implies lower $P(\text{flood})$**: the storm has not yet peaked, so there is genuine uncertainty about whether the peak will reach the alert level.

- Above the alert level, all states converge to $P \approx 1$ regardless of direction, as the evidence for flood breach is overwhelming.

The maximum asymmetry occurs near the alert threshold (4.3 m), where the directional signal is most informative: $\Delta P = -0.622$. The v1.1 model would output the "Steady" column for both rising and falling water.

6.4 Feature Importance

Averaged across all 40 gauges, the four features contribute:

Feature	Importance
`water_level`	0.6054
`hour`	0.2233
`delta_w`	0.1407
`delta2_w`	0.0307

Water level remains the dominant predictor (60.5%), as expected. The velocity feature (`delta_w`) captures 14.1% of model importance—a substantial signal that enables the rising/falling asymmetry. The acceleration feature (`delta2_w`) contributes 3.1%, providing marginal refinement for distinguishing decelerating rises (approaching peak) from accelerating surges.

Compared to v1.1, the hour feature's relative importance decreased from ∼14% to 22.3%—however, this is a redistribution effect: in absolute terms, the temporal signal is captured more precisely because `delta_w` absorbs some of the temporal variation previously attributed to `hour` alone.

6.5 Automated Test Results

Current unit test evidence is filed with the model record in ModelRisk (Test Results and Unit Test).

7 Sensitivity Analysis

Sensitivity of the classifier's AUC to key hyperparameters was analysed by varying each parameter independently around a baseline of 200 trees, depth 4 and learning rate 0.1. This is not the production configuration in Section 4: the analysis has not been rerun at production settings, because its generator has been retired, so the findings and tables below are historical.

Key findings:

- `n_estimators`: AUC stabilises above 100 rounds; diminishing returns beyond 200 (at depth 4 and learning rate 0.1).
- `max_depth`: Optimal at 4; deeper trees overfit the noise.
- `learning_rate`: Sweet spot at 0.1; aggressive rates (>0.2) increase variance.

Table 1: Classifier sensitivity to n_estimators (gauge GAUGE-082c, max_depth=4, lr=0.1).

n_estimators	AUC-ROC	Accuracy	
50	0.8010	0.7936	
100	0.8002	0.7917	
200	0.7965	0.7865	**Baseline**
500	0.7931	0.7824	

Table 2: Classifier sensitivity to max_depth (gauge GAUGE-082c, n_estimators=200, lr=0.1).

max_depth	AUC-ROC	Accuracy	
2	0.7974	0.7904	
3	0.7975	0.7878	
4	0.7965	0.7865	**Baseline**
5	0.7951	0.7840	
6	0.7928	0.7801	

Table 3: Classifier sensitivity to learning_rate (gauge GAUGE-082c, n_estimators=200, max_depth=4).

learning_rate	AUC-ROC	Accuracy	
0.0100	0.7950	0.7869	
0.0500	0.8005	0.7901	
0.1000	0.7965	0.7865	**Baseline**
0.2000	0.7902	0.7811	
0.5000	0.7794	0.7763	

Table 4: Classifier sensitivity to subsample fraction (gauge GAUGE-082c, n_estimators=200, max_depth=4, lr=0.1).

subsample	AUC-ROC	Accuracy	
0.5000	0.7972	0.7891	
0.6000	0.7976	0.7853	
0.7000	0.7971	0.7897	
0.8000	0.7965	0.7865	**Baseline**
0.9000	0.7985	0.7881	
1.00	0.7975	0.7862	

Table 5: Feature importance at production hyperparameters — v1.1 vs v1.2 (gauge GAUGE-082c).

Feature	v1.1	v1.2
water_level	0.8607	0.6260
hour	0.1393	0.2590
delta_w	—	0.0960
delta2_w	—	0.0190
AUC-ROC	0.7965	0.9720

8 Model Selection: Challenger Comparison

8.1 Open Question

Should an alternative classifier, such as a Random Forest, be considered in place of the GBM? What evidence shows that the GBM is the most accurate and most stable choice for $P(\text{flood})$?

The GBM was selected on the basis of in-sample AUC. That is not sufficient evidence for model selection: no challenger has been fitted on the same data, and the AUC was measured on a split that overstated out-of-sample performance (Section 8.4). This section sets out the comparison that would settle the question. The protocol is written so that the GBM *can fail*: if a challenger wins under the pre-agreed decision rule, the challenger is adopted.

8.2 Why Random Forest Is the Natural Challenger

Both models are tree ensembles on the same four features, so they differ only in how the ensemble is built:

	GBM (champion)	**Random Forest (challenger)**
Ensemble	Sequential, each tree fits the residual of the last	Independent deep trees on bootstrap samples, averaged
Error reduced	Mainly bias	Mainly variance
Tuning	Sensitive (learning rate, depth, trees)	Robust to defaults
Probabilities	Optimises log loss directly; usually well calibrated	Averaged votes; pulled away from 0 and 1, often needs re-calibration
Seed sensitivity	Low–moderate (`subsample=0.8`)	Low with many trees

Because the classifier output is used directly as a price ($C_h = N \times p_h \times d$), calibration matters more than ranking. This is the main reason to expect the GBM to win, but it has to be shown, not assumed.

8.3 Candidate Set

1. **GBM** — production configuration (`n_estimators=300`, `max_depth=3`, `learning_rate=0.05`, `subsample=0.8`, `min_samples_leaf=50`).
2. **GBM, tuned** — the same model given the same tuning budget as the challengers. If it beats the production GBM, the production settings, not the model family, need changing.
3. **Random Forest** — `n_estimators=500`, `max_features=2`, `min_samples_leaf` tuned.
4. **Random Forest, calibrated** — the same, wrapped in isotonic or Platt calibration fitted on an inner fold. This stops RF losing only because of calibration that a one-line fix would remove.

5. **Logistic regression** on the four features — a transparent baseline.

6. **Flood polynomial** — the production fallback, as the floor every model must beat.

Every tree model gets the same tuning budget (for example 30 random-search trials) inside the training folds, so the GBM does not win only because it has been tuned more.

8.4 Prerequisite: Remove Storm Leakage

Each storm contributes 168 hourly rows, and the label (*future maximum level* $\geq w_{\text{severe}}$) is shared by adjacent hours. Up to version 2.1 of this document the trainer split by row, using a stratified `train_test_split`, so hours from the same storm appeared in both training and test sets, and near-miss rows were scored too. The reported AUC therefore measured interpolation within storms already seen, not prediction on new storms. This leakage favours deep, low-bias learners and so would flatter the Random Forest more than the GBM.

The production trainer now holds out 20% of storms, stratified on whether the storm floods, keeps near-miss rows in training and records `split` in its result. If no such split is possible (for example, only one storm floods) it falls back to the row-level split and records `split = row`. Classifiers and metrics trained before this change keep the old split until they are retrained, when the fitted models also change. On one test-portfolio gauge the new split moved Brier from 0.0213 to 0.0258 and AUC from 0.988 to 0.990. AUC can hold up because the easy gains from leakage are offset by no longer scoring near-miss rows, so Brier and log loss are the better guide to the change.

For the comparison itself:

- Carry a `storm_id` with every training vector.
- Split by storm using `GroupKFold` (5 folds), so every storm is entirely in train or entirely in test.
- Add near-miss synthetic vectors to the *training* folds only; score on real storm vectors only.

8.5 Accuracy Metrics

Scored on held-out storms, per gauge and pooled:

Primary Brier score and log loss — proper scoring rules that reward correct probabilities, not just correct ranking.

Calibration Reliability diagram, expected calibration error, calibration slope and intercept.

Discrimination AUC (secondary, kept for continuity with the existing thresholds).

Business Stress P&L error on held-out storms: the hourly $\sum_h N p_h d$ path compared against the realised knock-out outcome of that storm.

8.6 Stability Tests

"Most stable" is tested in five ways. Each produces a number per model and per gauge.

1. **Seed stability.** Retrain each model with 20 random seeds on the same data. Measure the standard deviation of p on a fixed grid (the probability surface of Section 4) and of peak stress P&L for a reference storm set.

2. **Data stability.** Retrain on 50 bootstrap resamples *of storms* (not of rows). Report the spread of p on the fixed grid and the spread of the held-out Brier score.

3. **Catalogue stability.** Train on one generation of the storm catalogue and score on a regenerated catalogue (a new seed or new storm-control parameters). This is the retraining trigger in practice, so it is the most decision-relevant stability test. The benchmark tool does not regenerate the catalogue itself: run it once on each catalogue generation and compare the two result files.

4. **Shape stability.** $P(\text{flood})$ should not fall as the water level rises. Count monotonicity violations in `log_h_s` across the grid, and measure how rough the finite-difference sensitivity $\partial p/\partial w$ is. Step-like surfaces give noisy stress Greeks.

5. **Cross-gauge consistency.** The distribution of per-gauge metrics: a model that is excellent on most gauges but fails badly on a few is less stable than one that is uniformly good.

8.7 Statistical Comparison

- Per gauge: paired difference in Brier score (GBM minus challenger) on the same held-out storms, with a 95% confidence interval from a storm-level bootstrap.
- Across gauges: Wilcoxon signed-rank test on the per-gauge Brier differences.
- Stability metrics: report ratios (challenger dispersion / GBM dispersion) with bootstrap intervals.

8.8 Decision Rule

Agreed before the results are seen. The GBM is retained if all of the following hold:

1. Its pooled Brier score is no worse than the best challenger's by more than a set tolerance (for example 2%), and the Wilcoxon test does not show the challenger significantly better.

2. Its seed, bootstrap and catalogue dispersion are no greater than the challenger's (within 0.002 in probability s.d.).

3. It has no more monotonicity violations than the challenger (within 0.5 percentage points of the violation rate).

If a challenger is better on accuracy but worse on stability (or the reverse), the trade-off is recorded and the choice is escalated for model-risk sign-off rather than decided by the model owner. If the best challenger wins on all three, it replaces the GBM. With fewer than six gauges the Wilcoxon test has no power, so "significantly better" then means every gauge's storm-bootstrap interval favours the challenger.

8.9 Running the Benchmark

The comparison is implemented in the benchmark tool:

Code `tools/flood_classifier_challenger/`

Settings `config/models/_flood_challenger.py`

It builds each gauge's training set with the production vector builder, so it scores exactly the features and labels production trains on:

```
python -m tools.flood_classifier_challenger \
    --gauges all
python -m tools.flood_classifier_challenger \
    --gauges G1,G2 --seeds 20 --bootstraps 50
python -m tools.flood_classifier_challenger \
    --synthetic 3     # no storm data needed
```

Defaults are 1,000 storms per gauge, 5 storm folds, 6 tuning trials, 10 seeds and 20 storm bootstraps; a sign-off run uses 20 seeds and 50 bootstraps as above. Fits run in parallel on every core. The run writes `challenger/challenger_results.json` (every metric, the tuned parameters per fold and the probability surfaces) and the table below.

8.10 Results

Run: 2026-10-30, source `catalogue thames from PhysicalRisk-testdata`, 6 gauges scored (0 skipped), 500 storms per gauge, 5 storm folds, 6 tuning trials, 10 seeds, 20 storm bootstraps.

Model	Brier	Log loss	AUC	ECE	Seed s.d.	Boot. s.d.	Mono. viol.
GBM (production)	0.0204	0.0688	0.989	0.0078	0.0090	0.0353	8.3%
GBM (tuned)	0.0191	0.0644	0.990	0.0051	0.0000	0.0495	8.9%
Random Forest	0.0185	0.0631	0.990	0.0054	0.0052	0.0354	11.2%
Random Forest (calib.)	0.0199	0.0752	0.988	0.0061	0.0420	0.0610	5.3%
Logistic regression	0.0378	0.1438	0.921	0.0244	0.0000	0.0175	0.0%
Flood polynomial	0.0203	0.0822	0.979	0.0074	–	–	11.5%

Decision: best challenger by pooled Brier is Random Forest (GBM Brier 0.0204 vs 0.0185, gap 10.03%; Wilcoxon $p = 0.016$). Accuracy test failed, stability test failed, shape test passed: the result is **mixed** and is escalated for model-risk sign-off.

Gauge	Storms	Flood rate	GBM Brier	Best chall.	Δ (95% CI)
GAUGE-ef7f75ec	500	12.6%	0.0297	0.0270	+0.0027 [+0.0015, +0.0039]
GAUGE-066fc60e	500	7.3%	0.0188	0.0166	+0.0022 [+0.0010, +0.0034]
GAUGE-bc801f07	500	3.6%	0.0111	0.0102	+0.0009 [+0.0001, +0.0018]
GAUGE-3324d948	500	7.3%	0.0210	0.0189	+0.0022 [+0.0010, +0.0032]
GAUGE-acc1b9cc	500	6.5%	0.0208	0.0197	+0.0011 [+0.0001, +0.0022]

Gauge	Storms	Flood rate	GBM Brier	Best chall.	Δ (95% CI)
GAUGE-a03e456e	500	7.1%	0.0208	0.0187	+0.0021 [+0.0008, +0.0033]

Δ is GBM minus challenger per-storm Brier; positive favours the challenger.

Reading these results. This run is illustrative, not sign-off evidence: it uses the first six gauges of the 18 in the test portfolio, at 500 storms each and the default seed and bootstrap counts. The tuned GBM's zero seed s.d. does not show better stability: tuning chose `subsample` = 1 for the fold used in the stability test on all six gauges, so the fit has no randomness for the seed to act on. Its bootstrap s.d., which measures sensitivity to the storms sampled, is the higher of the GBM variants and above the Random Forest's. Tuned parameters also differ between folds, so its accuracy is not that of one fixed setting.

9 Model Limitations and Known Weaknesses

1. **Hydrograph-derived features only:** The four features (water level, hour, velocity, acceleration) are all derived from the local hydrograph. External features (rainfall intensity, upstream levels, antecedent soil moisture) could further improve discrimination but would require expanded data pipelines.
2. **No spatial correlation:** Each gauge is modelled independently. Correlated flood events across multiple gauges are not captured.
3. **Training noise regime:** The time-decaying noise is a modelling choice, not an empirical calibration. Different noise functions would yield different AUC profiles.
4. **Binary classification only:** The model predicts flood/no-flood. Flood severity (depth, velocity, duration) is not estimated.
5. **Stationarity assumed:** The classifier does not account for changing flood characteristics due to climate change or infrastructure modifications.
6. **Alert-level threshold:** Training uses the FloodAlert trigger level. Models trained on Warning or Severe thresholds would produce different classifiers.

10 Change History

Version	Date	Description
1.0	3-Mar-2026	Initial model documentation
1.1	3-Mar-2026	Balanced training data (1,000 storms), probability surface
1.2	5-Mar-2026	Four-feature model: velocity (Δw) and acceleration ($\Delta^2 w$); AUC 0.9189 $\rightarrow$ 0.9557
1.3	9-Mar-2026	Log-transformed feature space ($\ln(w/s)$, $\ln(t/T)$, log-velocity, log-acceleration); AUC 0.9557 $\rightarrow$ 0.9525 (marginal reduction, better calibration)
2.0	11-Mar-2026	**Multi-storm upgrade:** training data replaced with 10,000 StormSequence objects from MKM-SS-001; 1,680,000 vectors/gauge (10$\times$ v1.x); multi-peak hydrographs model antecedent-wetness dynamics; stressm classifiers stored in `data/output/stressm/`; AUC 0.9525 $\rightarrow$ $\approx$0.993 (+4.2%); scope expanded to all 52 Thames gauges
2.1	18-Mar-2026	Stressm pipeline refactored — classifier training now integrated into unified stressm pipeline with seasonal baselines from gaugehd monthly means
2.2	30-Oct-2026	Added challenger comparison (GBM against Random Forest and others); trainer now holds out whole storms; hyperparameter table corrected to match the code